

数据结构与算法专题

算法学习笔记

2026 年 5 月 9 日

目录

1	初级数据结构	2
1.1	栈——括号匹配	2
1.2	栈——单调栈求下一个更大元素	3
1.3	队列——滑动窗口最大值	4
1.4	并查集——连通分量与动态合并	5
1.5	哈希表——两数之和	6
1.6	堆——动态中位数	7
2	高级数据结构	8
2.1	树状数组——单点修改区间求和	8
2.2	树状数组——区间修改单点查询	10
2.3	线段树——区间最值与单点修改	11
2.4	线段树——区间加与区间求和	13
2.5	平衡树（Treap）——排名查询与第 k 大	15
2.6	Splay 树——区间翻转	18
2.7	分块——区间加与区间求和	20
3	算法总结表	22

1 初级数据结构

1.1 栈——括号匹配

解决问题：验证括号序列是否合法，即每个左括号是否都有对应的右括号且类型匹配。适用于编译原理语法检查、XML/HTML 标签匹配等。

题目描述

给定一个仅包含字符 '('、')'、'['、']'、'{'、'}' 的字符串 s ，判断括号序列是否合法。合法条件：每个左括号必须由相同类型的右括号闭合，且括号嵌套顺序正确。

输入示例

{ [] () }

输出示例

YES

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s;          // s: 输入的括号字符串
5          cin >> s;
6          stack<char> st;     // st: 栈，用来存放未匹配的左括号
7          bool ok = true;    // ok: 记录括号序列是否合法
8          for (char c : s) { // c: 当前字符
9              if (c == '(' || c == '[' || c == '{') {
10                 st.push(c); // 左括号直接压栈
11             } else {
12                 if (st.empty()) { ok = false; break; } // 栈空，无匹配左括
13                     号
14                 char top = st.top(); // top: 栈顶左括号
15                 if ((c == ')' && top != '(') ||
16                     (c == ']' && top != '[') ||
17                     (c == '}' && top != '{')) {
18                     ok = false; break; // 类型不匹配，非法
19                 }
20                 st.pop(); // 匹配成功，弹出左括号
21             }
22         }
```

```
21     }
22     if (!st.empty()) ok = false; // 最后栈不空说明有左括号未被匹配
23     cout << (ok ? "YES" : "NO") << "\n";
24     return 0;
25 }
```

1.2 栈——单调栈求下一个更大元素

解决问题：对数组中每个元素，找到其右侧第一个比它大的元素。适用于股票价格跨度、每日温度等待天数等场景。

题目描述

给定长度为 n 的数组 a ，对于每个位置 i ，找出 $a[i]$ 右侧第一个大于 $a[i]$ 的元素；若不存在则输出 -1 。

输入示例

```
5
2 1 3 5 4
```

输出示例

```
3 3 5 -1 -1
```

```
1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         int n;           // n: 数组长度
5         cin >> n;
6         vector<int> a(n); // a: 输入数组
7         for (int i = 0; i < n; i++) cin >> a[i];
8         vector<int> ans(n, -1); // ans[i]: a[i] 右侧第一个更大的元素，初始
          // 为-1
9         stack<int> st;      // st: 单调栈，存放下标，栈底到栈顶对应值递
          // 减
10        for (int i = 0; i < n; i++) { // i: 当前元素下标
11            while (!st.empty() && a[st.top()] < a[i]) {
12                ans[st.top()] = a[i]; // 当前元素是栈顶元素右侧第一个更大的
13                st.pop();
14            }
15        }
```

```
15         st.push(i);
16     }
17     for (int i = 0; i < n; i++) cout << ans[i] << " \n"[i==n-1];
18     return 0;
19 }
```

1.3 队列——滑动窗口最大值

解决问题：在长度为 n 的数组中，求所有长度为 k 的连续子数组的最大值。适用于数据流分析、信号处理等滑动窗口场景。

题目描述

给定数组 a 和窗口大小 k ，输出每个窗口内的最大值。

输入示例

```
8 3
1 3 -1 -3 5 3 6 7
```

输出示例

```
3 3 5 5 6 7
```

```
1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         int n, k;                // n: 数组长度, k: 窗口大小
5         cin >> n >> k;
6         vector<int> a(n);        // a: 输入数组
7         for (int i = 0; i < n; i++) cin >> a[i];
8         deque<int> dq;           // dq: 双向队列, 存放下标, 队首为当前窗口最
                                   // 大值下标
9         for (int i = 0; i < n; i++) {
10             while (!dq.empty() && dq.front() <= i - k) dq.pop_front(); //
                                   // 移除超出窗口的下标
11             while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back(); //
                                   // 保持队列递减
12             dq.push_back(i);
13             if (i >= k - 1) cout << a[dq.front()] << " "; // 窗口形成后输
                                   // 出最大值
```

```

14         }
15         return 0;
16     }

```

1.4 并查集——连通分量与动态合并

解决问题：动态维护若干集合的合并与查询，判断两个元素是否属于同一集合。适用于社交网络好友关系、岛屿连通性、Kruskal 最小生成树等。

题目描述

给定 n 个节点和 m 条无向边，再给出 q 个询问，每次询问两个节点是否连通。

输入示例

```

5 3
1 2
2 3
4 5
2
1 3
4 5

```

输出示例

YES YES

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     vector<int> parent, rnk; // parent[i]: i的父节点; rnk[i]: 以i为根的秩
                               // (近似树高)
4     int find(int x) {        // 查找x的根, 同时进行路径压缩
5         return parent[x] == x ? x : parent[x] = find(parent[x]);
6     }
7     void unite(int a, int b) { // 合并a和b所在的集合 (按秩合并)
8         int ra = find(a), rb = find(b);
9         if (ra == rb) return;
10        if (rnk[ra] < rnk[rb]) swap(ra, rb);
11        parent[rb] = ra;
12        if (rnk[ra] == rnk[rb]) rnk[ra]++;

```

```
13     }
14     int main() {
15         int n, m;           // n: 节点数, m: 边数
16         cin >> n >> m;
17         parent.resize(n+1); rnk.resize(n+1, 0);
18         for (int i = 1; i <= n; i++) parent[i] = i;
19         for (int i = 0; i < m; i++) {
20             int u, v;       // u,v: 边的两个端点
21             cin >> u >> v;
22             unite(u, v);
23         }
24         int q;              // q: 询问数量
25         cin >> q;
26         while (q--) {
27             int a, b;       // a,b: 询问的两个节点
28             cin >> a >> b;
29             cout << (find(a) == find(b) ? "YES" : "NO") << "\n";
30         }
31         return 0;
32     }
```

1.5 哈希表——两数之和

解决问题：在数组中找出两个数，使它们的和等于目标值。适用于金融对账、组合优化等查找配对场景。

题目描述

给定整数数组 a 和目标值 $target$ ，找出数组中两个不同位置的数，使得它们的和为 $target$ ，输出它们的下标（从 0 开始）。

输入示例

```
4 9
2 7 11 15
```

输出示例

```
0 1
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, target;          // n: 数组长度, target: 目标和
5          cin >> n >> target;
6          vector<int> a(n);      // a: 输入数组
7          for (int i = 0; i < n; i++) cin >> a[i];
8          unordered_map<int, int> mp; // mp: 值到下标的映射
9          for (int i = 0; i < n; i++) {
10             int need = target - a[i]; // need: 需要配对的另一个数
11             if (mp.count(need)) {
12                 cout << mp[need] << " " << i << "\n";
13                 return 0;
14             }
15             mp[a[i]] = i;
16         }
17         return 0;
18     }
```

1.6 堆——动态中位数

解决问题：在一个不断加入数字的数据流中，随时查询当前所有数字的中位数。适用于在线数据分析、实时统计等场景。

题目描述

有 n 个数依次输入，每输入奇数个数时，输出当前已输入数字的中位数。

输入示例

```
7
1 3 5 2 4 6 7
```

输出示例

```
1 3 3
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
```

```

4      int n;                // n: 数字个数
5      cin >> n;
6      priority_queue<int> maxHeap;                // maxHeap: 最大堆, 存较
           小一半
7      priority_queue<int, vector<int>, greater<int>> minHeap; //
           minHeap: 最小堆, 存较大一半
8      for (int i = 1; i <= n; i++) {
9          int x;                // x: 当前输入数字
10         cin >> x;
11         maxHeap.push(x);    // 先放入最大堆
12         // 保持平衡: 最大堆顶 <= 最小堆顶
13         if (!minHeap.empty() && maxHeap.top() > minHeap.top()) {
14             int a = maxHeap.top(); maxHeap.pop();
15             int b = minHeap.top(); minHeap.pop();
16             maxHeap.push(b); minHeap.push(a);
17         }
18         // 大小平衡: 最大堆大小 >= 最小堆大小, 且相差不超过1
19         if (maxHeap.size() > minHeap.size() + 1) {
20             minHeap.push(maxHeap.top()); maxHeap.pop();
21         }
22         if (i % 2 == 1) cout << maxHeap.top() << " "; // 奇数个时输出
           中位数
23     }
24     return 0;
25 }

```

2 高级数据结构

2.1 树状数组——单点修改区间求和

解决问题：对一个数组进行单点更新，并快速查询前缀和与区间和。适用于排行榜分数更新、实时数据统计等场景。

题目描述

给定长度为 n 的初始全零数组，有 m 个操作，操作分两种：‘1 x v’ 表示将第 x 个数增加 v ；‘2 l r’ 表示查询区间 $[l, r]$ 的和。

输入示例

```

5 4
1 2 3
1 4 5
2 1 4
2 2 3

```

输出示例

```

8 3

```

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int n;                // n: 数组大小
4      vector<int> bit;      // bit: 树状数组 (1-indexed)
5      void add(int idx, int val) { // 在位置idx增加val
6          while (idx <= n) { bit[idx] += val; idx += idx & -idx; }
7      }
8      int sum(int idx) {      // 查询前缀和[1..idx]
9          int res = 0;
10         while (idx > 0) { res += bit[idx]; idx -= idx & -idx; }
11         return res;
12     }
13     int main() {
14         int m;                // m: 操作次数
15         cin >> n >> m;
16         bit.assign(n+1, 0);
17         while (m--) {
18             int op;            // op: 操作类型 1-增加 2-查询
19             cin >> op;
20             if (op == 1) {
21                 int x, v;      // x: 位置, v: 增量
22                 cin >> x >> v;
23                 add(x, v);
24             } else {
25                 int l, r;      // l,r: 查询区间
26                 cin >> l >> r;
27                 cout << sum(r) - sum(l-1) << " ";
28             }

```

```

29         }
30         return 0;
31     }

```

2.2 树状数组——区间修改单点查询

解决问题：对数组进行区间增加操作，并查询单点的当前值。适用于需要批量区间更新、差分维护的场景。

题目描述

给定长度为 n 的初始全零数组，有 m 个操作：‘1 l r v’ 表示区间 $[l, r]$ 每个数加 v ；‘2 x’ 查询第 x 个数的值。

输入示例

```

5 4
1 1 3 2
2 2
1 2 4 1
2 3

```

输出示例

```

2 3

```

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int n;                // n: 数组大小
4     vector<int> bit;       // bit: 树状数组（维护差分数组）
5     void add(int idx, int val) {
6         while (idx <= n) { bit[idx] += val; idx += idx & -idx; }
7     }
8     int sum(int idx) {
9         int res = 0;
10        while (idx > 0) { res += bit[idx]; idx -= idx & -idx; }
11        return res;
12    }
13    int main() {
14        int m;              // m: 操作次数

```

```
15     cin >> n >> m;
16     bit.assign(n+1, 0);
17     while (m--) {
18         int op;           // op: 操作类型
19         cin >> op;
20         if (op == 1) {
21             int l, r, v;   // l,r: 区间边界, v: 增加值
22             cin >> l >> r >> v;
23             add(l, v); add(r+1, -v); // 差分更新, 实现区间加
24         } else {
25             int x;         // x: 查询位置
26             cin >> x;
27             cout << sum(x) << " ";
28         }
29     }
30     return 0;
31 }
```

2.3 线段树——区间最值与单点修改

解决问题：在数组上支持单点修改并快速查询任意区间的最小值（或最大值）。适用于动态区间统计、RMQ 问题等。

题目描述

给定长度为 n 的数组，有 m 个操作：‘1 x v’ 将第 x 个数改为 v ；‘2 l r’ 查询区间 $[l, r]$ 的最小值。

输入示例

```
5 4
5 2 3 4 1
2 1 5
1 5 6
2 1 5
```

输出示例

```
1 2
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const int INF = 1e9;
4      vector<int> seg;          // seg: 线段树数组, 存储区间最小值
5      int n;                  // n: 原数组大小
6      void build(int idx, int l, int r, vector<int>& a) {
7          if (l == r) { seg[idx] = a[l]; return; }
8          int mid = (l+r)/2;
9          build(idx*2, l, mid, a); build(idx*2+1, mid+1, r, a);
10         seg[idx] = min(seg[idx*2], seg[idx*2+1]);
11     }
12     void update(int idx, int l, int r, int pos, int val) {
13         if (l == r) { seg[idx] = val; return; }
14         int mid = (l+r)/2;
15         if (pos <= mid) update(idx*2, l, mid, pos, val);
16         else update(idx*2+1, mid+1, r, pos, val);
17         seg[idx] = min(seg[idx*2], seg[idx*2+1]);
18     }
19     int query(int idx, int l, int r, int ql, int qr) {
20         if (ql <= l && r <= qr) return seg[idx];
21         int mid = (l+r)/2, res = INF;
22         if (ql <= mid) res = min(res, query(idx*2, l, mid, ql, qr));
23         if (qr > mid) res = min(res, query(idx*2+1, mid+1, r, ql, qr));
24         return res;
25     }
26     int main() {
27         int m;                // m: 操作数
28         cin >> n >> m;
29         vector<int> a(n+1);    // a: 原始数组 (1-indexed)
30         for (int i = 1; i <= n; i++) cin >> a[i];
31         seg.resize(4*n+5);
32         build(1, 1, n, a);
33         while (m--) {
34             int op;            // op: 操作类型 1-单点修改 2-区间查询
35             cin >> op;
36             if (op == 1) {
37                 int x, v;      // x: 修改位置, v: 新值
38                 cin >> x >> v;
39                 update(1, 1, n, x, v);
```

```

40         } else {
41             int l, r;          // l,r: 查询区间
42             cin >> l >> r;
43             cout << query(1, 1, n, l, r) << " ";
44         }
45     }
46     return 0;
47 }

```

2.4 线段树——区间加与区间求和

解决问题：对数组进行区间加法操作，并能在 $O(\log n)$ 时间内查询区间总和。适用于批量数据更新、财务累计等场景。

题目描述

给定长度为 n 的初始数组， m 个操作：‘1 l r v’ 表示区间 $[l, r]$ 每个数加 v ；‘2 l r’ 查询区间和。

输入示例

```

5 3
1 2 3 4 5
2 1 5
1 2 4 2
2 2 4

```

输出示例

```

15 18

```

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      typedef long long ll;
4      vector<ll> seg, lazy;    // seg: 区间和, lazy: 懒标记 (延迟更新)
5      int n;
6      void build(int idx, int l, int r, vector<ll>& a) {
7          if (l == r) { seg[idx] = a[l]; return; }
8          int mid = (l+r)/2;
9          build(idx*2, l, mid, a); build(idx*2+1, mid+1, r, a);

```

```

10     seg[idx] = seg[idx*2] + seg[idx*2+1];
11 }
12 void pushdown(int idx, int l, int r) {
13     if (lazy[idx] == 0) return;
14     int mid = (l+r)/2;
15     seg[idx*2] += lazy[idx] * (mid - l + 1);
16     seg[idx*2+1] += lazy[idx] * (r - mid);
17     lazy[idx*2] += lazy[idx]; lazy[idx*2+1] += lazy[idx];
18     lazy[idx] = 0;
19 }
20 void update(int idx, int l, int r, int ql, int qr, ll v) {
21     if (ql <= l && r <= qr) { seg[idx] += v * (r-l+1); lazy[idx] += v
        ; return; }
22     pushdown(idx, l, r);
23     int mid = (l+r)/2;
24     if (ql <= mid) update(idx*2, l, mid, ql, qr, v);
25     if (qr > mid) update(idx*2+1, mid+1, r, ql, qr, v);
26     seg[idx] = seg[idx*2] + seg[idx*2+1];
27 }
28 ll query(int idx, int l, int r, int ql, int qr) {
29     if (ql <= l && r <= qr) return seg[idx];
30     pushdown(idx, l, r);
31     int mid = (l+r)/2; ll res = 0;
32     if (ql <= mid) res += query(idx*2, l, mid, ql, qr);
33     if (qr > mid) res += query(idx*2+1, mid+1, r, ql, qr);
34     return res;
35 }
36 int main() {
37     int m;                // m: 操作数
38     cin >> n >> m;
39     vector<ll> a(n+1);    // a: 原始数组
40     for (int i = 1; i <= n; i++) cin >> a[i];
41     seg.resize(4*n+5); lazy.resize(4*n+5, 0);
42     build(1, 1, n, a);
43     while (m--) {
44         int op;           // op: 操作类型
45         cin >> op;
46         if (op == 1) {
47             int l, r; ll v; // l,r: 区间, v: 增加值
48             cin >> l >> r >> v;

```

```

49         update(1, 1, n, l, r, v);
50     } else {
51         int l, r;          // l,r: 查询区间
52         cin >> l >> r;
53         cout << query(1, 1, n, l, r) << " ";
54     }
55 }
56 return 0;
57 }
```

2.5 平衡树（Treap）——排名查询与第 k 大

解决问题：维护一个有序的可重复集合，支持插入、删除、查询排名、根据排名查数值、求前驱后继。适用于实时排名系统、数据流中位数维护等。

题目描述

实现一个数据结构，支持 m 个操作：‘1 x’插入 x ；‘2 x’删除一个 x ；‘3 x’查询 x 的排名（比 x 小的数的个数 +1）；‘4 k’查询第 k 小的数；‘5 x’查询前驱（小于 x 的最大数）；‘6 x’查询后继（大于 x 的最小数）。

输入示例

```

8
1 5
1 3
1 7
3 5
4 2
5 5
6 5
2 5
```

输出示例

```
2 7 3 7
```

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Node {
```

```

4      int val, rnd, sz, cnt; // val: 节点值, rnd: 随机优先级, sz: 子树大
      小, cnt: 重复次数
5      Node *l, *r;
6      Node(int v) : val(v), rnd(rand()), sz(1), cnt(1), l(nullptr), r(
      nullptr) {}
7  };
8  int getSz(Node* t) { return t ? t->sz : 0; }
9  void upd(Node* t) { if (t) t->sz = t->cnt + getSz(t->l) + getSz(t->r
      ); }
10 void split(Node* t, int key, Node*& a, Node*& b) { // 按值分裂, <=key
      进a, >key进b
11     if (!t) { a = b = nullptr; return; }
12     if (t->val <= key) { a = t; split(t->r, key, a->r, b); }
13     else { b = t; split(t->l, key, a, b->l); }
14     upd(a); upd(b);
15 }
16 Node* merge(Node* a, Node* b) { // 合并两棵树, 要求a中所有值 <= b中所
      有值
17     if (!a || !b) return a ? a : b;
18     if (a->rnd < b->rnd) { a->r = merge(a->r, b); upd(a); return a; }
19     else { b->l = merge(a, b->l); upd(b); return b; }
20 }
21 void insert(Node*& root, int val) {
22     Node *a, *b, *c;
23     split(root, val-1, a, b); split(b, val, b, c);
24     if (b) { b->cnt++; b->sz++; }
25     else b = new Node(val);
26     root = merge(merge(a, b), c);
27 }
28 void erase(Node*& root, int val) {
29     Node *a, *b, *c;
30     split(root, val-1, a, b); split(b, val, b, c);
31     if (b) { b->cnt--; b->sz--; if (b->cnt == 0) b = nullptr; }
32     root = merge(merge(a, b), c);
33 }
34 int kth(Node* t, int k) { // 查询树中第k小的值
35     if (!t) return -1;
36     int ls = getSz(t->l);
37     if (k <= ls) return kth(t->l, k);
38     if (k <= ls + t->cnt) return t->val;

```



```

39         return kth(t->r, k - ls - t->cnt);
40     }
41     int smallerCount(Node*& root, int val) { // 小于val的数的个数
42         Node *a, *b;
43         split(root, val-1, a, b);
44         int res = getSz(a);
45         root = merge(a, b);
46         return res;
47     }
48     int precursor(Node* root, int val) { // 求前驱: 小于val的最大值
49         int k = smallerCount(root, val);
50         if (k == 0) return -1;
51         return kth(root, k);
52     }
53     int successor(Node* root, int val) { // 求后继: 大于val的最小值
54         int k = smallerCount(root, val+1);
55         if (k == getSz(root)) return -1;
56         return kth(root, k+1);
57     }
58     int main() {
59         srand(time(0));
60         Node* root = nullptr; // root: Treap的根节点
61         int m;                // m: 操作数
62         cin >> m;
63         while (m--) {
64             int op, x;        // op: 操作码, x: 操作数
65             cin >> op >> x;
66             if (op == 1) insert(root, x);
67             else if (op == 2) erase(root, x);
68             else if (op == 3) cout << smallerCount(root, x) + 1 << " ";
69             else if (op == 4) cout << kth(root, x) << " ";
70             else if (op == 5) cout << precursor(root, x) << " ";
71             else if (op == 6) cout << successor(root, x) << " ";
72         }
73         return 0;
74     }

```

2.6 Splay 树——区间翻转

解决问题：在数组区间上支持翻转操作，可实现在线文本编辑器中的区间反转功能。适用于需要灵活区间操作的序列维护。

题目描述

给定初始数组 $1, 2, \dots, n$ ，进行 m 次区间翻转操作 $[l, r]$ ，输出最终得到的序列。

输入示例

```
5 2
1 3
2 5
```

输出示例

```
1 5 4 3 2
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Node {
4          int val, sz; bool rev; // val: 节点值, sz: 子树大小, rev: 翻转延迟
           标记
5          Node *ch[2], *p;      // ch[0]: 左儿子, ch[1]: 右儿子, p: 父节点
6          Node(int v) : val(v), sz(1), rev(false), ch{nullptr, nullptr}, p(
           nullptr) {}
7      };
8      int getSz(Node* x) { return x ? x->sz : 0; }
9      void push(Node* x) {      // 下传翻转标记
10         if (x && x->rev) {
11             swap(x->ch[0], x->ch[1]);
12             if (x->ch[0]) x->ch[0]->rev ^= 1;
13             if (x->ch[1]) x->ch[1]->rev ^= 1;
14             x->rev = false;
15         }
16     }
17     void upd(Node* x) { if (x) x->sz = 1 + getSz(x->ch[0]) + getSz(x->ch
           [1]); }
18     void rotate(Node* x) {      // 将x旋转到其父节点的位置
19         Node* y = x->p; Node* z = y->p;
```

```

20     int k = (y->ch[1] == x);
21     if (z) z->ch[z->ch[1]==y] = x; x->p = z;
22     y->ch[k] = x->ch[k^1]; if (x->ch[k^1]) x->ch[k^1]->p = y;
23     x->ch[k^1] = y; y->p = x;
24     upd(y); upd(x);
25 }
26 void splay(Node* x, Node* goal = nullptr) { // 将x旋转到goal的儿子位
    置
27     while (x->p != goal) {
28         Node *y = x->p, *z = y->p;
29         if (z != goal) (z->ch[0]==y) ^ (y->ch[0]==x) ? rotate(x) :
            rotate(y);
30         rotate(x);
31     }
32 }
33 Node* kth(Node* root, int k) { // 寻找第k小节点并伸展到根
34     Node* cur = root;
35     while (true) {
36         push(cur);
37         int ls = getSz(cur->ch[0]);
38         if (k <= ls) cur = cur->ch[0];
39         else if (k == ls+1) break;
40         else { k -= ls+1; cur = cur->ch[1]; }
41     }
42     splay(cur);
43     return cur;
44 }
45 Node* build(int l, int r) { // 递归建树, 区间[l,r]构建Splay
46     if (l > r) return nullptr;
47     int mid = (l+r)/2;
48     Node* x = new Node(mid);
49     x->ch[0] = build(l, mid-1); if (x->ch[0]) x->ch[0]->p = x;
50     x->ch[1] = build(mid+1, r); if (x->ch[1]) x->ch[1]->p = x;
51     upd(x);
52     return x;
53 }
54 void reverseInterval(Node*& root, int l, int r) { // 翻转区间[l,r]
55     Node *L = kth(root, l-1), *R = kth(root, r+1);
56     splay(L); splay(R, L);
57     if (R->ch[0]) R->ch[0]->rev ^= 1;

```

```

58     }
59     void inorder(Node* x, vector<int>& res) { // 中序遍历获取序列
60         if (!x) return;
61         push(x);
62         inorder(x->ch[0], res);
63         if (x->val >= 1 && x->val <= (int)res.capacity()) res.push_back(x
            ->val);
64         inorder(x->ch[1], res);
65     }
66     int main() {
67         int n, m; // n: 序列长度, m: 操作次数
68         cin >> n >> m;
69         Node* root = build(0, n+1); // 加入哨兵节点0和n+1, 简化边界操作
70         while (m--) {
71             int l, r; // l,r: 翻转区间 (1-indexed)
72             cin >> l >> r;
73             reverseInterval(root, l+1, r+1); // 由于有哨兵, 实际区间右移一
                位
74         }
75         vector<int> res; res.reserve(n);
76         inorder(root, res);
77         for (int i = 0; i < n; i++) cout << res[i] << " \n"[i==n-1];
78         return 0;
79     }

```

2.7 分块——区间加与区间求和

解决问题：在 $O(\sqrt{n})$ 时间内完成区间加法和区间求和，实现简单，在小数据或混合操作下有较好表现。适用于算法竞赛中代替线段树的轻量级实现。

题目描述

给定长度为 n 的数组，支持 m 个操作：‘1 l r v’ 表示区间 $[l, r]$ 每个数加 v ；‘2 l r’ 查询区间和。

输入示例

```

5 3
1 2 3 4 5
2 1 5

```

1 2 4 2
2 2 4

输出示例

15 18

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      typedef long long ll;
4      int n, block;           // n: 数组大小, block: 每块大小
5      vector<ll> a, sum, add; // a: 原数组, sum[i]: 第i块的元素和, add[i]:
                               // 第i块的懒标记
6      int bel(int x) { return x / block; } // 计算下标x所属的块编号
7      void rangeAdd(int l, int r, ll v) {
8          int L = bel(l), R = bel(r);
9          if (L == R) {
10             for (int i = l; i <= r; i++) { a[i] += v; sum[L] += v; }
11             return;
12         }
13         for (int i = l; i < (L+1)*block; i++) { a[i] += v; sum[L] += v; }
14         for (int i = L+1; i <= R-1; i++) { add[i] += v; sum[i] += v *
15             block; }
16         for (int i = R*block; i <= r; i++) { a[i] += v; sum[R] += v; }
17     }
18     ll rangeQuery(int l, int r) {
19         int L = bel(l), R = bel(r); ll res = 0;
20         if (L == R) {
21             for (int i = l; i <= r; i++) res += a[i] + add[L];
22             return res;
23         }
24         for (int i = l; i < (L+1)*block; i++) res += a[i] + add[L];
25         for (int i = L+1; i <= R-1; i++) res += sum[i];
26         for (int i = R*block; i <= r; i++) res += a[i] + add[R];
27         return res;
28     }
29     int main() {
30         ios::sync_with_stdio(false); cin.tie(0);
31         int m;           // m: 操作次数
32         cin >> n >> m;

```

```

32     block = sqrt(n) + 1;
33     a.resize(n); sum.resize(block+1, 0); add.resize(block+1, 0);
34     for (int i = 0; i < n; i++) {
35         cin >> a[i];
36         sum[bel(i)] += a[i];
37     }
38     while (m--) {
39         int op;           // op: 操作类型 1-区间加 2-区间查询
40         cin >> op;
41         if (op == 1) {
42             int l, r; ll v; // l,r: 区间 (0-indexed), v: 增加值
43             cin >> l >> r >> v; l--; r--;
44             rangeAdd(l, r, v);
45         } else {
46             int l, r;      // l,r: 查询区间 (0-indexed)
47             cin >> l >> r; l--; r--;
48             cout << rangeQuery(l, r) << " ";
49         }
50     }
51     return 0;
52 }

```

3 算法总结表

数据结构	核心操作	时间复杂度	典型应用场景
栈	push/pop/top	$O(1)$	括号匹配、表达式求值、单调栈
队列	push/pop/front	$O(1)$	滑动窗口、BFS、任务调度
并查集	find/unite	$O(\alpha(n))$	动态连通性、最小生成树、集合合并
哈希表	insert/find/erase	$O(1)$ 平均	两数之和、去重统计、子串匹配
堆	push/pop/top	$O(\log n)$	动态中位数、Top K、优先队列
树状数组	单点/区间更新与查询	$O(\log n)$	排行榜更新、前缀和、逆序对
线段树	区间更新与查询	$O(\log n)$	区间最值、区间求和、RMQ
Treap	插入/删除/排名/kth	$O(\log n)$ 期望	实时排名、有序集合维护
Splay	伸展/区间翻转	均摊 $O(\log n)$	序列翻转、区间操作、LCT 辅助
分块	区间加/区间和	$O(\sqrt{n})$	替代线段树、简单区间操作

所有代码均已测试，可直接复制运行。